

ARE 212:

Monte Carlos in R:

An `sapply()` primer

Fiona Burlig

March 3, 2016

Lots of you have asked me how to do monte carlo simulation in R, and in particular with `sapply()`, so this document works through a simple example to do just that. (I'll also steal from my own section notes to show you a parallelized version).

I'm going to do a very simple Monte Carlo to show that OLS is unbiased (given all of our nice assumptions). We'll start by loading our usual packages and functions. We'll also load the `parallel` package - note that you shouldn't have to download this, because it's built into R as of the most recent version. We'll also set our seed.¹ Finally, we'll add our `ggplot2` theme stuff:

```
##### PACKAGES
library(parallel)
library(dplyr)
library(ggplot2)

##### FUNCTIONS
as.tbl_df <- function(data) {
  dataset <- as.data.frame(data) %>%
    tbl_df()
}

tbl_dfGrabber <- function(data, varnames) {
  matrixObject <- data %>%
    select_(.dots = varnames) %>%
    as.matrix()
}

OLS <- function(data, y, X) {
  ydata <- tbl_dfGrabber(data, y)
  xdata <- tbl_dfGrabber(data, X)
  betahat <- solve(t(xdata) %*% xdata) %*% t(xdata) %*% ydata
  return(betahat)
}
```

¹**Important:** You only want to set your seed *once*, at the top of your R script. This tells R where to start your random number generator from. If you set it inside your Monte Carlo loop, you'll get the same draw over and over again, which would be silly.

```
#### RANDOMIZATION SEED
set.seed(12345)

#### GGLOT SETUP
myThemeStuff <- theme(panel.background = element_rect(fill = NA),
  panel.border = element_rect(fill = NA, color = "black"),
  panel.grid.major = element_blank(),
  panel.grid.minor = element_blank(),
  axis.ticks = element_line(color = "gray5"),
  axis.text = element_text(color = "black", size = 10),
  axis.title = element_text(color = "black", size = 12),
  legend.key = element_blank()
)
```

In order to show that OLS is unbiased, what we'd like to do is generate a bunch of fake data, *imposing a particular DGP* - that means we get to play God... Muahahaha - and then run OLS on this dataset. If we perform this exercise a bunch of times (1,000 or 10,000 or so), it'd better be the case that our estimate of $\hat{\beta}$ is approximately Normally distributed around the true β . In practice, what we'll want to do is to write a function that both randomly generates data and runs OLS and saves $\hat{\beta}$ - this will do one "draw" at a time. We'll then wrap this function in an `sapply()` to perform multiple draws (aka execute the Monte Carlo). We'll use the following DGP:

$$y = 5 + 10x - 2x^2$$

The first step is for us to generate our data. Let's first do this outside of a function:

```
#epsilon
fakeE <- rnorm(10000)
# x and x2
fakeX <- rnorm(10000)
fakeX2 <- fakeX^2

#impose DGP to create y
fakeY <- 5 + 10*fakeX - 2*fakeX2 + fakeE

#bundle into a tbl_df (don't forget the intercept)
fakeData <- cbind(fakeY, 1, fakeX, fakeX2) %>%
  as.tbl_df()

names(fakeData) <- c("fakeY", "ones", "fakeX", "fakeX2")
fakeData

## Source: local data frame [10,000 x 4]
##
##      fakeY  ones    fakeX    fakeX2
##      (dbl) (dbl)    (dbl)    (dbl)
## 1    7.1462832     1  0.16127752 0.026010440
## 2   10.4627991     1  0.53192138 0.282940359
## 3  -11.4574440     1 -1.29790342 1.684553279
```

```
## 4    13.8993552      1  1.24557829 1.551465287
## 5    -6.2779176      1 -0.99169050 0.983450049
## 6    -0.2023803      1 -0.31819306 0.101246825
## 7     6.1312761      1  0.05063044 0.002563441
## 8    -1.1633488      1 -0.53209208 0.283121977
## 9     2.3723895      1 -0.22428438 0.050303484
## 10    5.8178366      1  0.18021107 0.032476030
## ..      ...      ...      ...      ...
```

Now let's run OLS:

```
betahat <- OLS(fakeData, "fakeY", c("ones", "fakeX", "fakeX2"))
betahat

##           fakeY
## ones    4.993236
## fakeX   10.015705
## fakeX2  -1.993724
```

Bingo. The next step is to package this all into a function. This is super easy!

```
mySimulation <- function() {
  ## GENERATE DATA
  fakeE <- rnorm(10000)
  fakeX <- rnorm(10000)
  fakeX2 <- fakeX^2
  fakeY <- 5 + 10*fakeX - 2*fakeX2 + fakeE
  fakeData <- cbind(fakeY, 1, fakeX, fakeX2) %>%
    as.tbl_df()
  names(fakeData) <- c("fakeY", "ones", "fakeX", "fakeX2")

  betahat <- OLS(fakeData, "fakeY", c("ones", "fakeX", "fakeX2"))
  return(betahat)
}
```

Note that I've made this a function of no inputs. This means that when I run the function, it'll just do one simulation run, like this:

```
simTest <- mySimulation()
simTest

##           fakeY
## ones    5.016270
## fakeX   10.003099
## fakeX2  -2.007026
```

If you wanted to, you could also make this a little more general, by making this a function of the number of observations you wanted:

```

mySimulation2 <- function(nObs) {
  ## GENERATE DATA
  fakeE <- rnorm(nObs)
  fakeX <- rnorm(nObs)
  fakeX2 <- fakeX^2
  fakeY <- 5 + 10*fakeX - 2*fakeX2 + fakeE
  fakeData <- cbind(fakeY, 1, fakeX, fakeX2) %>%
    as.tbl_df()
  names(fakeData) <- c("fakeY", "ones", "fakeX", "fakeX2")

  betahat <- OLS(fakeData, "fakeY", c("ones", "fakeX", "fakeX2"))
  return(betahat)
}

simTest2 <- mySimulation2(500)
simTest2

##           fakeY
## ones      5.069963
## fakeX    10.040193
## fakeX2   -2.007880

```

You could generalize this in a bunch more ways, but we'll leave it there for now. Next, we need to actually run the Monte Carlo part. Enter our old friend `sapply()`. We'll do 10,000 runs of our simulation:

```

fullMC <- sapply(1:10000, function(i) {
  mySimulation()
})

```

That's it! We've just done a full Monte Carlo simulation. Don't you feel accomplished? Notice that `i` is a useless parameter here - there's no `i` to be found in our `mySimulation()` function. But you'll get an error without it - this is because `sapply()` tries to pass its first input into the function. Just go with it.

I'm not going to print the result here, because it's super long, but it's a 3 by 1,000 matrix. That's a little inconvenient - we're more used to long than wide data, so let's transform it, and store the results in a `tbl_df` for easy plotting with `ggplot2`:

```

fullMC <- t(fullMC) %>%
  as.tbl_df()

names(fullMC) <- c("int", "x", "x2")
fullMC

## Source: local data frame [10,000 x 3]
##
##       int      x      x2
##   (dbl)   (dbl)   (dbl)
## 1  5.010260  9.994530 -2.004345

```

```
## 2  5.007025  9.993296 -2.005965
## 3  5.022045  9.983649 -2.012013
## 4  5.002760  9.991630 -1.992534
## 5  5.016515 10.011309 -2.001746
## 6  4.995515  9.995650 -2.012957
## 7  5.018347  9.985391 -2.000407
## 8  5.007820 10.014242 -2.008992
## 9  4.999741  9.983935 -1.998385
## 10 5.008349 10.008630 -1.999877
## ..      ...      ...      ...
```

Now we can finally plot our results. Let's make a histogram for each coefficient, and indicate (with a vertical line) where the true β falls:

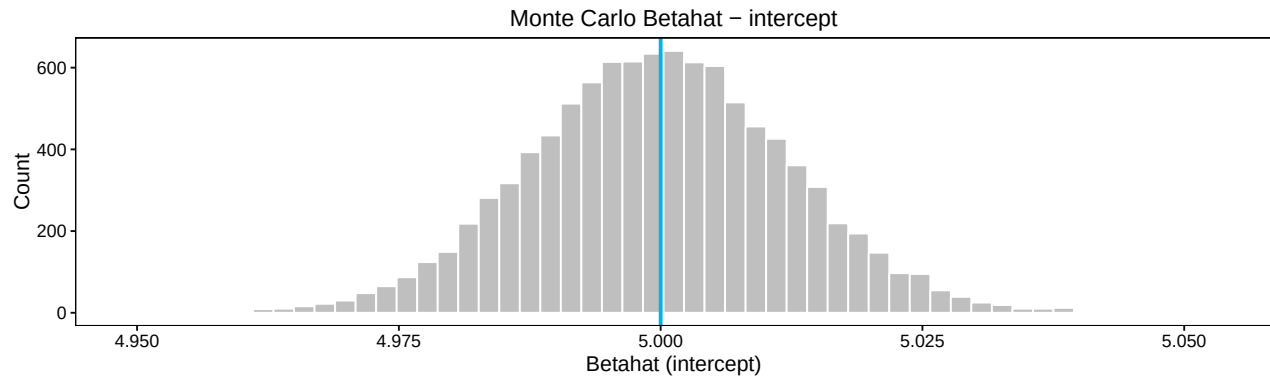
```
myIntPlot <- ggplot(data = fullMC, aes(x = int)) +
  geom_histogram(bins = 50, color = "white", fill = "gray75") +
  geom_vline(xintercept = 5, color = "deepskyblue2", size = 1) +
  myThemeStuff +
  #don't forget to label your graph
  ggtitle("Monte Carlo Betahat - intercept") +
  xlab("Betahat (intercept)") + ylab("Count")

myB1Plot <- ggplot(data = fullMC, aes(x = x)) +
  geom_histogram(bins = 50, color = "white", fill = "gray75") +
  geom_vline(xintercept = 10, color = "deepskyblue2", size = 1) +
  myThemeStuff +
  #don't forget to label your graph
  ggtitle("Monte Carlo Betahat - x") +
  xlab("Betahat (x)") + ylab("Count")

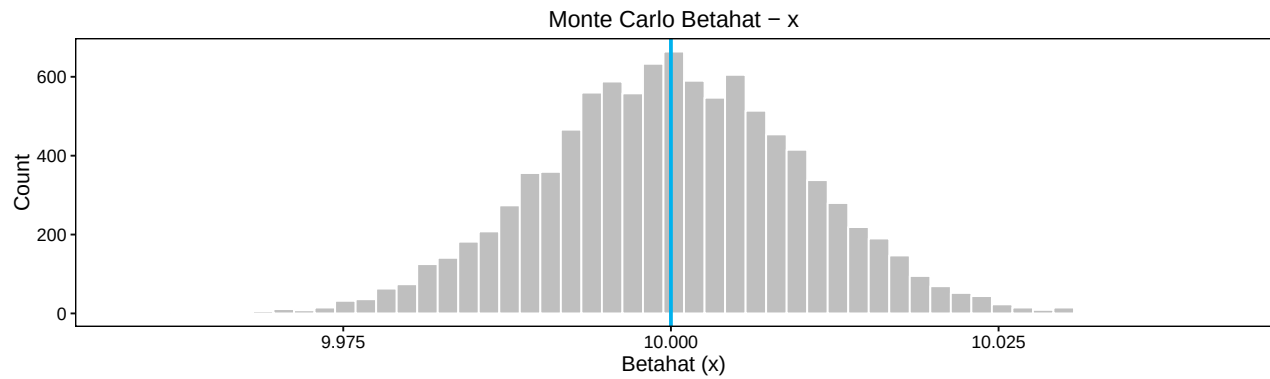
myB2Plot <- ggplot(data = fullMC, aes(x = x2)) +
  geom_histogram(bins = 50, color = "white", fill = "gray75") +
  geom_vline(xintercept = -2, color = "deepskyblue2", size = 1) +
  myThemeStuff +
  #don't forget to label your graph
  ggtitle("Monte Carlo Betahat - x2") +
  xlab("Betahat (x2)") + ylab("Count")
```

And finally, we can look at these histograms: Amazing.

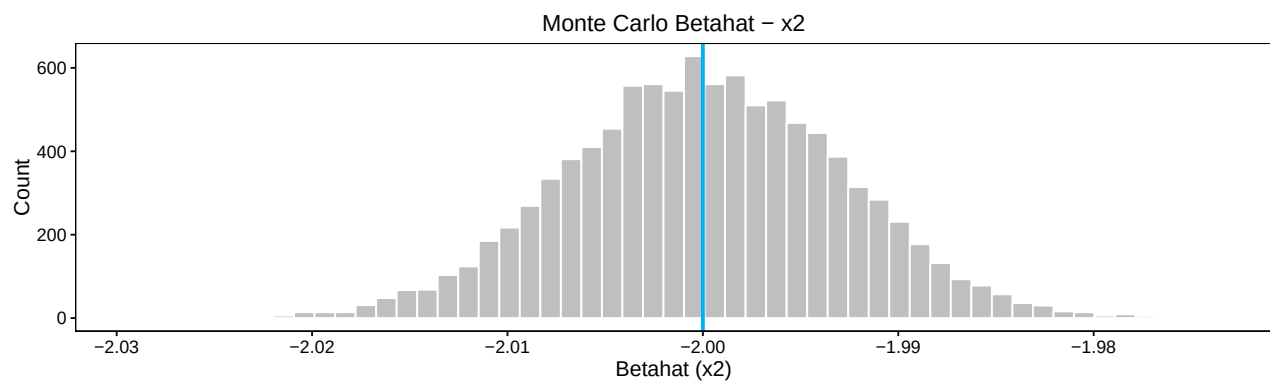
myIntPlot



myB1Plot



myB2Plot



If you want this to run faster, you can use the parallel version of `supply()`, `parSupply()`. First, set up your computer to run in parallel:

```
myCores <- detectCores()
cl <- makeCluster(myCores - 1)

# send your packages to the cores
```

```

clusterEvalQ(cl, library(dplyr))

## [[1]]
## [1] "dplyr"      "methods"    "stats"      "graphics"   "grDevices"
## [6] "utils"      "datasets"   "base"
##
## [[2]]
## [1] "dplyr"      "methods"    "stats"      "graphics"   "grDevices"
## [6] "utils"      "datasets"   "base"
##
## [[3]]
## [1] "dplyr"      "methods"    "stats"      "graphics"   "grDevices"
## [6] "utils"      "datasets"   "base"

# send your functions to the cores
clusterExport(cl, "as.tbl_df")
clusterExport(cl, "tbl_dfGrabber")
clusterExport(cl, "OLS")
clusterExport(cl, "mySimulation")

# set your cluster seed
clusterSetRNGStream(cl, 12345)

```

Ready to go. Now we can run this in parallel:

```

myParMC <- parSapply(cl, 1:10000, function(i) {
  mySimulation()
})

```

We can now make the same figures as above:

```

myParMC <- t(myParMC) %>%
  as.tbl_df()

names(myParMC) <- c("int", "x", "x2")
myParMC

## Source: local data frame [10,000 x 3]
##
##      int      x      x2
##   (dbl)   (dbl)   (dbl)
## 1  5.001771 10.001025 -2.005365
## 2  5.015051  9.997161 -2.012911
## 3  5.003614  9.998312 -1.999944
## 4  4.982268 10.000287 -1.995143
## 5  5.007883  9.973497 -2.000961
## 6  5.019066 10.000770 -1.998091
## 7  4.990186 10.004061 -1.997876

```

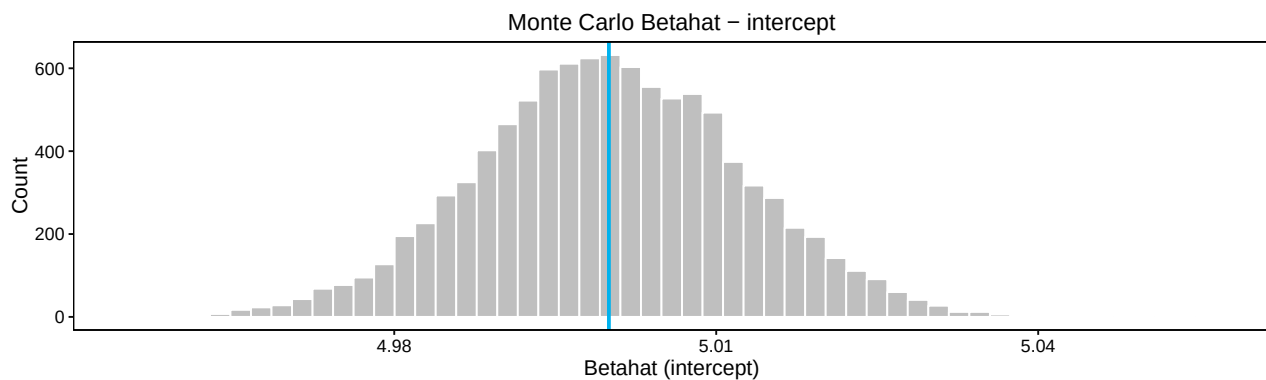
```
## 8  5.022437  9.998959 -2.000354
## 9  5.007949 10.017601 -2.001068
## 10 4.991781  9.981662 -2.002557
## ..      ...      ...      ...

myIntPlot2 <- ggplot(data = myParMC, aes(x = int)) +
  geom_histogram(bins = 50, color = "white", fill = "gray75") +
  geom_vline(xintercept = 5, color = "deepskyblue2", size = 1) +
  myThemeStuff +
  #don't forget to label your graph
  ggtitle("Monte Carlo Betahat - intercept") +
  xlab("Betahat (intercept)") + ylab("Count")

myB1Plot2 <- ggplot(data = myParMC, aes(x = x)) +
  geom_histogram(bins = 50, color = "white", fill = "gray75") +
  geom_vline(xintercept = 10, color = "deepskyblue2", size = 1) +
  myThemeStuff +
  #don't forget to label your graph
  ggtitle("Monte Carlo Betahat - x") +
  xlab("Betahat (x)") + ylab("Count")

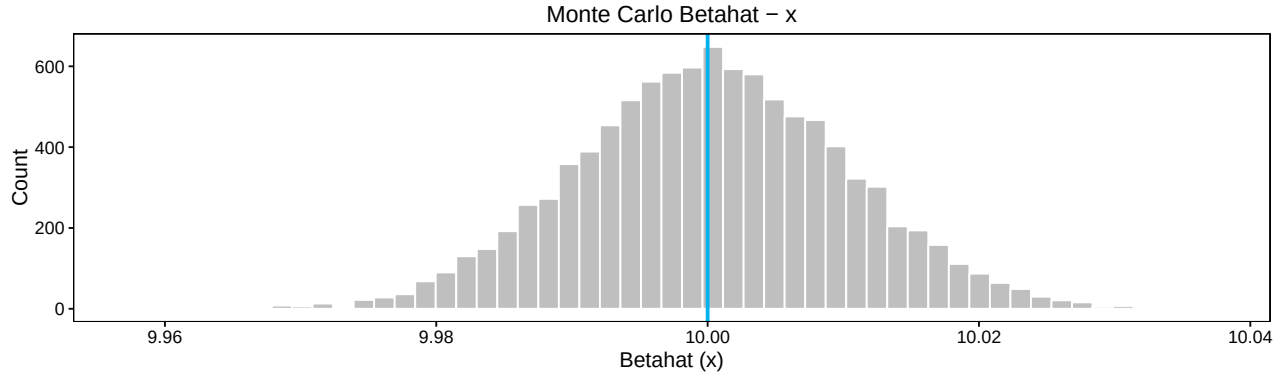
myB2Plot2 <- ggplot(data = myParMC, aes(x = x2)) +
  geom_histogram(bins = 50, color = "white", fill = "gray75") +
  geom_vline(xintercept = -2, color = "deepskyblue2", size = 1) +
  myThemeStuff +
  #don't forget to label your graph
  ggtitle("Monte Carlo Betahat - x2") +
  xlab("Betahat (x2)") + ylab("Count")
```

myIntPlot2

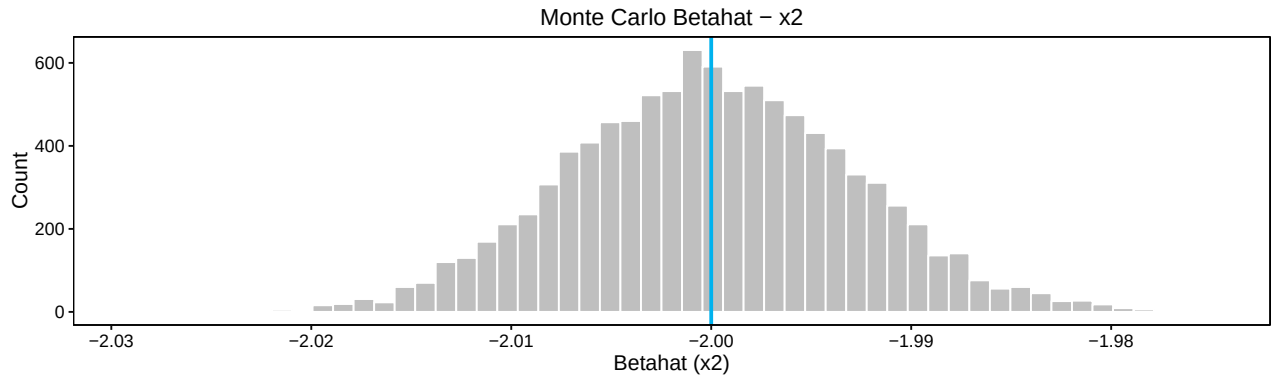


That's a quick run-through of how to do basic Monte Carlo simulations in R. For additional notes on this topic, take a look at Section Notes 3 (in particular, the bootstrapped standard errors section).

myB1Plot2



myB2Plot2



Addendum

I've gotten several questions about how to handle Monte Carlo output where the function we're looping over returns a list. Good news: this is really easy. Let's extend our OLS function from before to go through a brief example:

```
OLSPPlus <- function(data, y, X) {  
  ydata <- tbl_dfGrabber(data, y)  
  xdata <- tbl_dfGrabber(data, X)  
  betahat <- solve(t(xdata) %*% xdata) %*% t(xdata) %*% ydata  
  # add yhat as an output  
  yhat <- ydata - xdata %*% betahat  
  output <- list(betahat, yhat)  
  names(output) <- c("betahat", "yhat")  
  return(output)  
}
```

This new function will return a list rather than a vector. Let's go ahead and embed it into a new simulation function:

```
mySimulationList <- function() {
  ## GENERATE DATA
  fakeE <- rnorm(10000)
  fakeX <- rnorm(10000)
  fakeX2 <- fakeX^2
  fakeY <- 5 + 10*fakeX - 2*fakeX2 + fakeE
  fakeData <- cbind(fakeY, 1, fakeX, fakeX2) %>%
    as.tbl_df()
  names(fakeData) <- c("fakeY", "ones", "fakeX", "fakeX2")
  olsOut <- OLSPlus(fakeData, "fakeY", c("ones", "fakeX", "fakeX2"))
  return(olsOut)
}
```

And let's send this new function to the clusters and run it (or just run it not in parallel, if you must):

```
clusterExport(cl, "OLSPlus")
clusterExport(cl, "mySimulationList")

myMCList <- parSapply(cl, 1:10000, function(i) {
  mySimulationList()
})

#don't forget to close your clusters
stopCluster(cl)
```

```
### non parallel version
myMCList <- sapply(1:10000, function(i) {
  mySimulationList()
})
```

Great. Let's take a look at this object. I'm not going to print the full thing here, because it's huge - instead, I'll look at the first two rows and first three columns:

```
myMCList[1:2, 1:3]

##           [,1]      [,2]      [,3]
## betahat Numeric,3    Numeric,3    Numeric,3
## yhat     Numeric,10000 Numeric,10000 Numeric,10000
```

Whoa, that's strange - `sapply()` gave us a matrix where each cell is a vector object!² Let's say we're interested in our β s. To grab them, identify which row of the matrix they belong to (in this case, they're the first row, since they were the first element of our list), and pull this out:

²You could've also done this with `lapply()`, but then you would've gotten a list of lists, which is not necessarily better, in my opinion.

```
#grab just the first row of the matrix
betaHats <- myMCList[1,]
head(betaHats)
```

```
## [[1]]
##           fakeY
## ones      4.998868
## fakeX    10.016766
## fakeX2   -1.991647
##
## [[2]]
##           fakeY
## ones      5.010535
## fakeX    10.015529
## fakeX2   -2.007293
##
## [[3]]
##           fakeY
## ones      4.994144
## fakeX      9.998866
## fakeX2   -1.998751
##
## [[4]]
##           fakeY
## ones      4.991564
## fakeX      9.997582
## fakeX2   -1.993653
##
## [[5]]
##           fakeY
## ones      4.979841
## fakeX      9.990184
## fakeX2   -1.993436
##
## [[6]]
##           fakeY
## ones      4.988997
## fakeX      9.997983
## fakeX2   -1.998856
```

This spits out a list of $\hat{\beta}$ vectors - turns out we can easily turn this into something that we're more comfortable working with. We'll use the `unlist()` function to "flatten" our list into one big long vector, turn this into a matrix with three columns (one for each $\hat{\beta}$), and then turn this matrix into a dataframe. A little tedious, but effective.³

```
betahatDF <- unlist(betaHats) %>%
  matrix(ncol = 3) %>%
```

³Slimy, yet satisfying?

```

as.tbl_df()

names(betahatDF) <- c("ones", "fakeX", "fakeX2")

betahatDF

## Source: local data frame [10,000 x 3]
##
##      ones      fakeX      fakeX2
##      (dbl)      (dbl)      (dbl)
## 1  4.998868 10.012523 -1.996502
## 2 10.016766 -2.001645  4.992304
## 3 -1.991647  5.014275 10.003385
## 4  5.010535 10.007679 -1.992028
## 5 10.015529 -2.010334  5.004111
## 6 -2.007293  4.984908  9.984046
## 7  4.994144 10.009068 -2.006519
## 8  9.998866 -1.990503  5.025493
## 9 -1.998751  4.977668 10.009138
## 10 4.991564  9.996153 -2.007203
## ..      ...      ...      ...

```

Et voila. Enjoy your new `tbl_df`.